

Transferencia de Aprendizaje

Preentrenar, adaptar: finetuning, destilación, prompting y adaptación de dominio

Sergio Hernández, PhD
shernandez@ucm.cl



Magíster en Data Science — Universidad Católica del Maule



Redes Profundas

- ▶ Requieren miles (o millones) de ejemplos etiquetados.
- ▶ Se entrenan **desde cero** para cada problema.
- ▶ Etiquetar es caro: médicos, agrónomos, expertos.

Seres Humanos

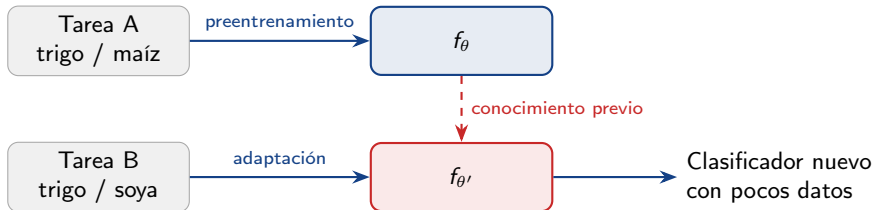
- ▶ **One-shot:** aprendemos de un solo ejemplo.
- ▶ **Few-shot:** bastan unos pocos ejemplos.
- ▶ **Zero-shot:** resolvemos problemas nuevos sin ninguna adaptación.

Idea Central

La **transferencia de aprendizaje** reutiliza conocimiento previo para resolver problemas nuevos con pocos datos, en lugar de partir desde cero cada vez.

Escenario: un Dron Agrícola

Un dron fue entrenado para clasificar **trigo vs. maíz**. Este año el agricultor cambia de cultivo: ahora debe distinguir **trigo vs. soya**.



Pregunta

¿Podemos aprovechar el clasificador existente f_{θ} para aprender la nueva tarea más rápido y con menos ejemplos?

Dos Etapas: Preentrenamiento y Adaptación

1. Preentrenamiento

- ▶ Adquirir **conocimiento previo**.
- ▶ Datos masivos y genéricos (ImageNet, LAION, texto web).
- ▶ Costoso, pero se hace **una sola vez**.

2. Adaptación

- ▶ Aplicar ese conocimiento a una **tarea nueva**.
- ▶ Pocos datos, específicos del dominio.
- ▶ Barato: minutos u horas.

¿Qué se puede transferir?

- ▶ **Pesos** de la red → finetuning, heads y probes.
- ▶ **Salidas** de un modelo maestro → destilación.
- ▶ **Entradas** modificadas → prompting, adaptación de dominio.
- ▶ **Datos** sintéticos → modelos generativos.

Finetuning: Ajuste Fino

Inicializar el nuevo clasificador con los parámetros preentrenados y **continuar entrenando** en la nueva tarea.

Tres Pasos

1. Preentrenar $f : \mathcal{X} \rightarrow \mathcal{Y}$.
2. Inicializar $f' = f$.
3. Ajustar $f' : \mathcal{X}' \rightarrow \mathcal{Y}'$ con descenso de gradiente.

Algoritmo

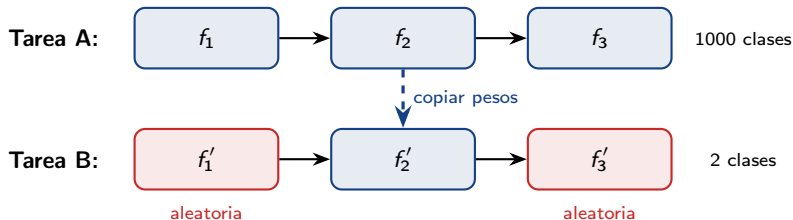
1. $\theta \leftarrow \arg \min_{\theta} \sum_i \mathcal{L}(f_{\theta}(x^{(i)}), y^{(i)})$
2. $\theta' \leftarrow \theta$ (inicialización)
3. $\theta' \leftarrow \arg \min_{\theta'} \sum_j \mathcal{L}(f_{\theta'}(x'^{(j)}), y'^{(j)})$

Por qué funciona

El paso 3 parte de un punto del espacio de parámetros que ya codifica características útiles (bordes, texturas, partes de objetos). La optimización converge más rápido y generaliza mejor con pocos datos.

Arquitecturas Incompatibles: Reemplazar Capas

Si la nueva tarea cambia la dimensión de entrada o salida (p.ej. 1000 clases $\rightarrow$ 2 clases), no podemos copiar la red completa.

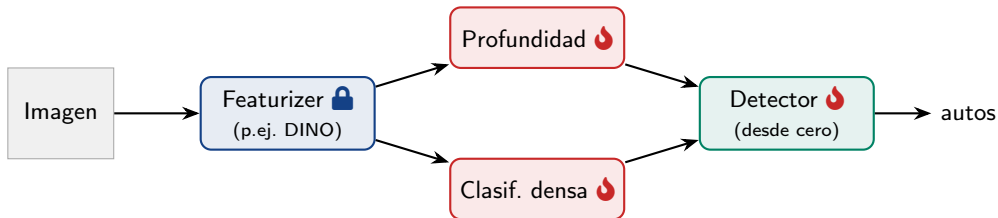


$$f = f_1 \circ f_2 \circ f_3 \quad \longrightarrow \quad f' = f'_1 \circ f'_2 \circ f'_3, \quad f'_2 \leftarrow f_2$$

Lo usual: conservar el **cuerpo** (backbone) y reemplazar la **cabeza** (última capa).

Finetuning de Subpartes de una Red

No es necesario ajustar todo: algunos módulos se **congelan** 🔒 y otros se **actualizan** 🔥.



¿Por qué congelar?

- ▶ **Componer** modelos que ya resuelven subtarefas.
- ▶ Evitar el **olvido** del conocimiento previo.
- ▶ Menos parámetros $\Rightarrow$ menos sobreajuste y menos memoria.

Ajuste de Bajo Rango (LoRA)

En lugar de ajustar $W \in \mathbb{R}^{d \times k}$ completo:

$$W' = W + BA, \quad B \in \mathbb{R}^{d \times r}, \quad A \in \mathbb{R}^{r \times k}, \quad r \ll d, k$$

Ahorra memoria y **regulariza** la adaptación.

Heads y Probes Lineales

Un **probe** es un módulo de lectura liviano h_ϕ que toma características de la red congelada y produce predicciones.

$$\phi^* = \arg \min_{\phi} \mathbb{E}_{x,y} \left[\mathcal{L}(h_\phi(f_\theta(x)), y) \right], \quad h_\phi(z) = Wz + b$$

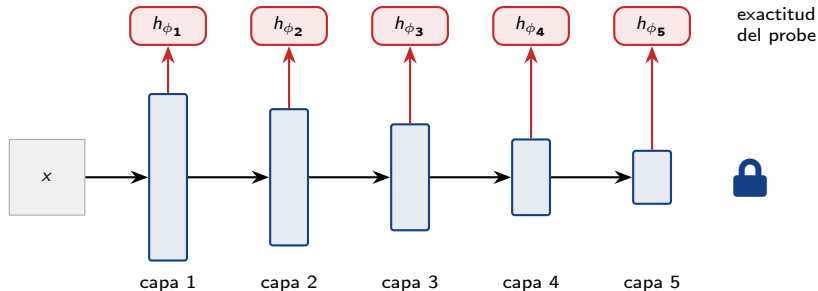
Ventajas

- ▶ Muy pocos parámetros.
- ▶ Problema convexo: a menudo con **solución cerrada** (mínimos cuadrados) o regresión logística.
- ▶ Las características se calculan una sola vez.

Doble Propósito

- ▶ **Adaptación:** reutilizar una red para una tarea nueva.
- ▶ **Análisis:** medir *qué* sabe cada capa. Si un probe lineal predice profundidad desde la capa ℓ , esa capa codifica profundidad.

Probes por Capa: ¿Dónde Está el Conocimiento?



Patrón típico en CNNs

- ▶ Capas tempranas: bordes, colores, texturas → **genéricas**, se transfieren bien.
- ▶ Capas profundas: partes y conceptos → **específicas** de la tarea de preentrenamiento.
- ▶ Por eso se reemplazan/ajustan las últimas capas y se reutilizan las primeras.

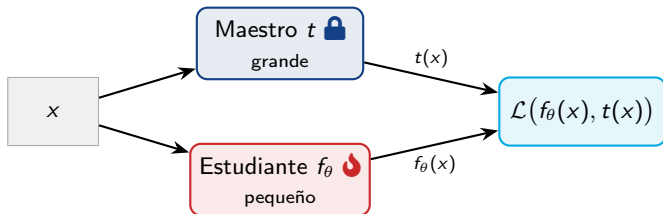
	Dominio similar	Dominio distinto
Pocos datos	Probe lineal sobre características congeladas.	Probe sobre capas intermedias; ajuste cuidadoso (LoRA).
Muchos datos	Finetuning de toda la red con tasa de aprendizaje baja.	Finetuning completo (el preentrenamiento sigue siendo buena inicialización).

Consejos

- ▶ Usar una **tasa de aprendizaje menor** en el backbone que en la cabeza nueva.
- ▶ Entrenar primero la cabeza (backbone congelado) y luego descongelar.
- ▶ Aplicar el mismo preprocesamiento (normalización, tamaño) que en el preentrenamiento.

Aprender de un Maestro

Hasta ahora aprendemos **de ejemplos** (x, y) . Otra opción: aprender de un **modelo maestro** $t : \mathcal{X} \rightarrow \mathcal{Y}$ ya entrenado.



Dos Modos de Aprender del Maestro

1. **Imitar las salidas** del maestro.
2. **Igualar las activaciones intermedias** del maestro.

El aprendizaje supervisado es un caso particular: el “maestro” es el mundo, que sólo entrega la etiqueta final.

Destilación (Hinton et al., 2015): el estudiante imita la distribución de probabilidad del maestro.

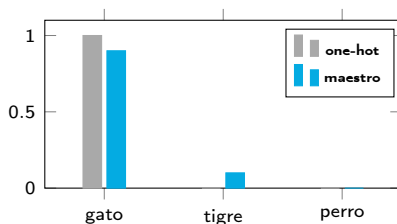
$$t, f_{\theta} : \mathcal{X} \rightarrow \Delta^{K-1}, \quad \theta^* = \arg \min_{\theta} \mathbb{E}_x [\mathcal{L}(f_{\theta}(x), t(x))]$$

Etiqueta Real vs. Maestro

Imagen de un gato doméstico:

- ▶ Etiqueta: [gato=1, tigre=0, perro=0]
- ▶ Maestro: [gato=0.9, tigre=0.1, perro=0]

Las **etiquetas blandas** revelan la estructura de similitud: un gato se parece más a un tigre que a un perro.



Destilación: Temperatura y Pérdida

Para exponer mejor las probabilidades pequeñas, se suaviza el softmax con una **temperatura** $T > 1$:

$$p_k^{(T)} = \frac{\exp(z_k/T)}{\sum_j \exp(z_j/T)}, \quad \mathcal{L} = (1 - \alpha) \mathcal{L}_{CE}(y, f_\theta(x)) + \alpha T^2 \text{KL}(t^{(T)}(x) \| f_\theta^{(T)}(x))$$

Por qué funciona

- ▶ Cada ejemplo aporta K números, no sólo uno $\Rightarrow$ señal más rica.
- ▶ Puede acelerar el aprendizaje y requerir menos datos.
- ▶ No necesita etiquetas: basta con imágenes sin anotar.

Usos

- ▶ **Compresión:** modelo grande $\rightarrow$ modelo móvil (DistilBERT, TinyViT).
- ▶ **Ensembles:** destilar varios modelos en uno.
- ▶ Hoy el término se usa para cualquier método estudiante-maestro.

Prompting: Adaptar Editando la Entrada

Prompting: resolver un problema nuevo **modificando la entrada**, sin cambiar los parámetros del modelo.

En Lenguaje Natural

Se antepone una instrucción al texto:

"Responde en francés: ¿qué hora es?"

El mismo modelo resuelve tareas distintas según el prompt.

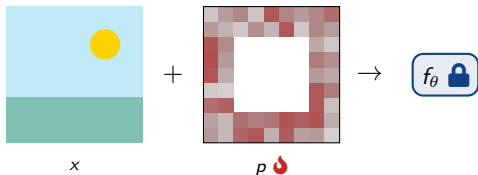
En Visión

- ▶ **Prompts de texto** para modelos visión-lenguaje (CLIP: "una foto de un {clase}").
- ▶ **Prompts visuales:** agregar píxeles o tokens a la entrada.
- ▶ Combinación por concatenación $\oplus$, suma, u otra mezcla.

$$\epsilon^* = \arg \min_{\epsilon} \mathbb{E}_{x,y} \left[\mathcal{L}(f_{\theta}(x \oplus \epsilon), y) \right] \quad (\text{sólo se aprende el prompt})$$

Prompting en el Espacio de Píxeles

Se aprende un patrón p que se **suma** a la imagen: $x + p$. Típicamente un **borde de ruido** optimizado.



$$p^* = \arg \min_p \sum_i \mathcal{L}(f_\theta(x^{(i)} + p), y^{(i)})$$

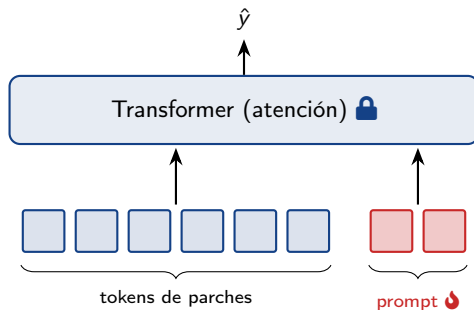
Un prompt distinto por tarea: clasificación de escenas, estética, objetos, ...

Relación con Ejemplos Adversarios

Matemáticamente es casi idéntico a un ataque adversario (*adversarial reprogramming*), pero el ruido se optimiza para **mejorar** el desempeño en la nueva tarea, no para degradarlo.

Prompting con Tokens en Vision Transformers

En un ViT, se concatenan **tokens aprendibles** de dimensión d a los tokens de parches.



- ▶ El prompt se mezcla con los parches vía **atención**: modula las transformaciones aplicadas a la imagen.
- ▶ Sólo se entrenan $d \times (\text{\#tokens})$ parámetros (*Visual Prompt Tuning*).
- ▶ A diferencia de las CNN, el transformer acepta entradas de **longitud variable**.

Modelos Entrenados para Ser “Promptables”

Un modelo puede entrenarse explícitamente para aceptar prompts **interpretables por humanos**.

Texto → Imagen

- ▶ DALL-E, Stable Diffusion.
- ▶ Instrucción en lenguaje natural condiciona la generación.

Segmentación

- ▶ Segment Anything (SAM).
- ▶ Prompt: clic, caja o trazo dibujado por el usuario.

Zero-shot

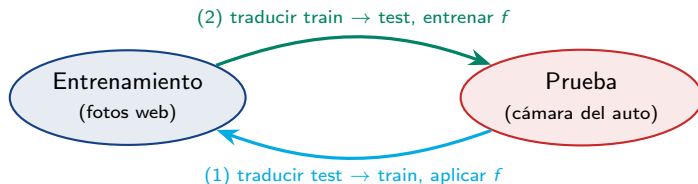
- ▶ CLIP: clasificar con nombres de clases nunca vistos.
- ▶ Sin ningún ejemplo etiquetado.

Cambio de Paradigma

El usuario adapta el modelo **en tiempo de uso**, sin gradientes ni reentrenamiento: la adaptación se vuelve interactiva.

Problema: la distribución de entrenamiento difiere de la de prueba, $p_{\text{train}}(x) \neq p_{\text{test}}(x)$.

Ejemplo: modelo entrenado con fotos de internet, desplegado en la cámara de un auto autónomo (lluvia, noche, otro sensor).



Estrategia

Hacer que ambas distribuciones sean **idénticas**: adaptar el dominio de prueba para que parezca el de entrenamiento, o viceversa.

Adaptación de Dominio No Supervisada

Datos Disponibles

- ▶ Entrenamiento etiquetado: $\{(x^{(i)}, y^{(i)})\}_{i=1}^N$ del dominio fuente.
- ▶ Prueba **sin etiquetas**: $\{x'^{(j)}\}_{j=1}^M$ del dominio objetivo.
- ▶ No se pueden usar etiquetas de prueba, pero sí **alinear distribuciones**.

Algoritmo: Traducción Adversaria

1. Entrenar un traductor G (GAN, p.ej. CycleGAN) tal que $G(x)$ sea indistinguible de x' para un discriminador D .
2. Traducir el conjunto de entrenamiento: $\{(G(x^{(i)}), y^{(i)})\}$ (las etiquetas se conservan).
3. Entrenar el clasificador f sobre los datos traducidos.
4. Aplicar f directamente en el dominio de prueba.

$$\min_G \max_D \mathbb{E}_{x'} [\log D(x')] + \mathbb{E}_x [\log (1 - D(G(x)))]$$

Relación con Prompting

Ambos **editan la entrada**. Pero la adaptación de dominio busca alinear distribuciones (no optimizar la tarea) y no requiere acceso al modelo.

Datos Generativos: Transferir la Distribución

Los datos sintéticos de un modelo generativo **no son sólo** una copia barata de datos reales.

Cantidad

- ▶ Muestras **ilimitadas**.
- ▶ Interpolan y remezclan los datos de entrenamiento de formas nuevas.

Calidad

- ▶ Cada muestra viene con su **procedimiento generativo**.
- ▶ Las variables latentes z tienen estructura explotable.

Ejemplo: Vacas en la Playa

Un generador entrenado con imágenes al aire libre nunca vio vacas en la playa. **Editando** z se generan esas combinaciones y se entrena un clasificador robusto al contexto.

Tipo de Conocimiento Transferido

Se transfiere conocimiento sobre la **distribución de los datos**, no una función aprendida: el clasificador final puede entrenarse desde cero.

Casi **cualquier componente** de un algoritmo de aprendizaje puede preentrenarse y transferirse.

Pérdida Aprendida

- ▶ Preentrenar la propia función $\mathcal{L}$.
- ▶ Ej.: pérdidas perceptuales (LPIPS), discriminadores de GAN.

Optimizador Aprendido

- ▶ Preentrenar la regla de actualización:

$$\theta_{t+1} = \theta_t - g_{\psi}(\nabla_{\theta} \mathcal{L}, \dots)$$

- ▶ ψ se aprende sobre muchas tareas.

Ejercicio Mental

Mire cada línea del código de entrenamiento (datos, modelo, pérdida, optimizador, inicialización) y pregúntese: ¿esto podría preentrenarse?

Un Catálogo Combinatorio de Métodos

Todos los métodos son **minimización del riesgo empírico**; difieren en **qué se adapta** y **qué queda fijo del preentrenamiento**.

Método	Objetivo de aprendizaje	Inferencia
Finetuning	$\arg \min_{\theta} \mathbb{E}_{x,y} [\mathcal{L}(f_{\theta}(x), y)], \theta_0 = \theta$	$f_{\theta}(x)$
Destilación	$\arg \min_{\theta} \mathbb{E}_x [\mathcal{L}(f_{\theta}(x), t(x))]$	$f_{\theta}(x)$
Prompting	$\arg \min_{\epsilon} \mathbb{E}_{x,y} [\mathcal{L}(f_{\theta}(x \oplus \epsilon), y)]$	$f_{\theta}(x \oplus \epsilon)$
Probes / heads	$\arg \min_{\phi} \mathbb{E}_{x,y} [\mathcal{L}(h_{\phi}(f_{\theta}(x)), y)]$	$h_{\phi}(f_{\theta}(x))$
Pérdida aprendida	$\arg \min_{\theta} \mathbb{E}_{x,y} [\mathcal{L}(f_{\theta}(x), y)]$	$f_{\theta}(x)$
Optimizador aprendido	$\arg \min_{\theta} \mathbb{E}_{x,y} [\mathcal{L}(f_{\theta}(x), y)]$	$f_{\theta}(x)$

Rojo: se adapta en la nueva tarea. **Azul:** preentrenado y fijo. (La adaptación de dominio no encaja limpiamente en este esquema.)

Los Métodos se Combinan

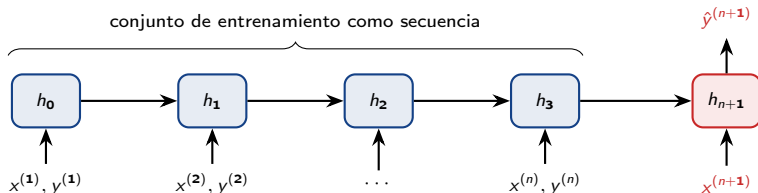
- ▶ Prompting + probe: tokens aprendidos y cabeza lineal sobre un ViT congelado.
- ▶ Destilación + finetuning: inicializar el estudiante con pesos preentrenados.
- ▶ LoRA + datos generativos: ajustar un modelo de difusión y usarlo como fuente de datos.

¿Qué Elegir?

	Parámetros nuevos	Acceso a pesos	Costo
Finetuning completo	todos	sí	alto
LoRA / subpartes	pocos	sí	medio
Probe lineal	muy pocos	sólo features	bajo
Prompting	muy pocos	gradientes / caja negra	bajo
Destilación	modelo nuevo	sólo salidas	medio

Modelos Secuenciales como Adaptación

Un modelo secuencial (RNN, Transformer) **cambia su comportamiento** según lo que ya ha visto: eso es adaptación en tiempo de ejecución.



Aprendizaje Supervisado con una RNN

- ▶ Entrada: los pares de entrenamiento en orden $\{x^{(1)}, y^{(1)}, \dots, x^{(n)}, y^{(n)}\}$.
- ▶ El estado oculto h_n "resume" los datos: la función aprendida está en (θ, h_n) .
- ▶ Una consulta nueva $x^{(n+1)}$ produce $\hat{y}^{(n+1)}$ **sin ningún gradiente**.

Aprender = Adaptar el Comportamiento Futuro según Datos Pasados

- ▶ Si entrenamos el modelo secuencial sobre **muchas tareas**, aprende a aprender: **metaaprendizaje**.
- ▶ En los grandes modelos de lenguaje esto emerge solo y se llama **aprendizaje en contexto** (*in-context learning*).

```
perro → dog  
gato → cat  
vaca → cow  
caballo → ?
```

Significado

El “aprendizaje” emerge del modelado de secuencias, sin un procedimiento de entrenamiento explícito. El prompt **es** el conjunto de entrenamiento.

- ▶ **Preentrenar + adaptar** es el paradigma dominante: casi nadie entrena desde cero.
- ▶ Lo que se transfiere puede ser:
 - ▶ **Pesos** → finetuning, LoRA, probes lineales.
 - ▶ **Salidas** → destilación de conocimiento.
 - ▶ **Entradas** → prompting, adaptación de dominio.
 - ▶ **Datos** → modelos generativos como fuente de entrenamiento.
- ▶ Los métodos son **combinables**: una caja de herramientas sobre la misma minimización del riesgo empírico.
- ▶ Los modelos secuenciales muestran que la adaptación puede ocurrir **sin gradientes** (aprendizaje en contexto).
- ▶ Mientras no existan modelos universales *zero-shot*, la adaptación seguirá siendo esencial, y su importancia crece con la escala de los modelos.